Linked Lists: Breaking the Chains of Fixed Memory

A deep dive into one of Computer Science's most elegant data structures — from intuitive mental models to production-ready C++ and Python code.

PART 1 OF 5

30 SLIDES

PART 1 — PHILOSOPHY OF LINKED DATA

What Is a Linked List?

🗺️ The Treasure Hunt Analogy

Imagine a treasure hunt where you only receive the **first clue** at the start. Each location you visit holds two things: a piece of **treasure (the data)** and **directions to the next location (the pointer)**. You can't skip ahead — you must follow the chain, one clue at a time, until you reach a dead end (NULL).

💻 The CS Definition

A **Linked List** is a linear data structure where elements, called **nodes**, are stored in memory at non-contiguous locations. Each node contains:

- **Data** — the value being stored
- **Pointer/Reference** — the address of the next node

The list begins at the **head** node and terminates when a node's pointer is **NULL** (or **None** in Python).

Array vs. Linked List

🥚 Array = Egg Carton

Fixed, contiguous slots reserved at creation. You know exactly where slot 4 is — but adding a 13th egg to a 12-carton is impossible without getting a new, bigger carton. **Size is set at compile/init time.**

- O(1) random access by index
- Expensive insertions/deletions (shifting)
- Fixed or costly resizing

🕺 Linked List = Conga Line

People (nodes) can **join or leave the line at any point** without everyone else shifting. The line naturally grows and shrinks. You just grab someone's shoulder (a pointer) and off you go.

- O(N) access — must traverse from head
- O(1) insertion/deletion at known position
- Dynamically sized at runtime

The Anatomy of a Node

Think of a **train car**: it carries cargo (data) and has a hitch connecting it to the next car (the pointer). Remove the hitch, and the rest of the train is lost!

Data Field

Holds the actual value: an integer, string, object, or any type. This is the "cargo" of the train car — the reason the node exists.

Next Pointer

Stores the **memory address** of the next node. In C++, this is a raw pointer (`Node*`). In Python, it's an object reference. Points to `NULL`/`None` at the tail.

The Node Together

Nodes are **self-referential structures** — a struct or class that contains a pointer/reference to its own type. This is what makes the chain possible and extensible.

```
// Conceptual structure
struct Node {
 int data; // The cargo
 Node* next; // The hitch to the next car
};
```

Memory Layout: C++ vs. Python

⚙️ C++ — Explicit & Manual

In C++, **you** are the memory manager. Nodes live on the **heap**, allocated with `new` and freed with `delete`. Raw pointers store exact memory addresses. Forgetting to `delete` causes **memory leaks**.

```
Node* n = new Node();
n->data = 42;
n->next = nullptr;
// Later: delete n;
```

🐍 Python — Automatic & Clean

Python handles memory **under the hood**. Objects live on the heap, but the **garbage collector** frees them automatically when there are no more references. No `new`, no `delete` — just assign and go.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
# Memory freed automatically
```

Key Insight: The logic of linked lists is identical in both languages. The difference is purely in **who manages memory** — you (C++) or the runtime (Python).

PART 2 — SINGLY LINKED LISTS

Introduction to Singly Linked Lists (SLL)

🚗 One-Way Street Analogy

A Singly Linked List is like a **one-way street**. Traffic flows in one direction only — from the **head** toward the **NULL terminator**. Once you pass a node, you cannot go back without starting over from the head. There are no U-turns.

This simplicity makes SLLs lightweight and efficient for forward-only workflows like stacks, queues, and simple iteration.

📐 SLL Structure at a Glance

- Each node has exactly **one pointer**: `next`
- Head points to the **first node**
- Tail's `next` points to **NULL**
- Traversal: **O(N)** in all cases
- Space per node: **data + 1 pointer**

Defining the SLL Node — Code

The node is the atomic unit of every linked list. Here's how it's defined in both C++ and Python — same concept, different syntax.

C++ — Struct with Pointer

```
struct Node {
    int data;
    Node* next;

    // Constructor
    Node(int val) {
        data = val;
        next = nullptr;
    }
};
```

Uses a `struct` (or `class`). The `next` field is a raw pointer (`Node*`) initialized to `nullptr`.

Python — Class with __init__

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Uses a `class` with `__init__`. The `next` attribute is set to `None` (Python's equivalent of `nullptr`). No explicit type declarations needed.

Notice: Both structures hold a data value and a reference to the next node. The philosophical structure is **identical** — only the syntax differs.

Traversing the SLL

🍞 **Breadcrumb Analogy:** Following breadcrumbs through a forest — you pick up one, look where it points, move there, and repeat until there are no more crumbs (NULL/None).

C++ Traversal

```
void printList(Node* head) {
    Node* p = head;
    while (p != nullptr) {
        cout << p->data << " -> ";
        p = p->next; // Move forward
    }
    cout << "NULL" << endl;
}
```

Python Traversal

```
def print_list(head):
    p = head
    while p is not None:
        print(p.data, end=" -> ")
        p = p.next  # Move forward
    print("None")
```

Start at Head

Create a temp pointer `p = head`. Never move `head` itself!

Check for NULL

Loop while `p != nullptr`. This is our stopping condition.

Advance Pointer

Move forward: `p = p->next`. The list advances one node.

SLL Insertion at the Head

🎟️ VIP Cutting the Line

Inserting at the head is like a **VIP guest cutting straight to the front** of a line. The new person steps in front of everyone, and the old first person becomes second. Fast, O(1), no traversal needed.

Order matters critically: First point the new node to the current head, *then* update head. Reversing this order loses the list!

Code: Insert at Head

```
// C++
void insertHead(Node*& head, int val) {
    Node* newNode = new Node(val);
    newNode->next = head; // Step 1
    head = newNode;       // Step 2
}

# Python
def insert_head(head, val):
    new_node = Node(val)
    new_node.next = head  # Step 1
    head = new_node       # Step 2
    return head
```

SLL Insertion at the Tail

🛒 **Checkout Line Analogy:** You walk to the back of a checkout line and stand behind the last person. You must **walk the entire line** to find the last person first — there's no shortcut.

C++ — Insert at Tail

```
void insertTail(Node*& head, int val) {
    Node* newNode = new Node(val);
    if (!head) { head = newNode; return; }
    Node* p = head;
    while (p->next != nullptr)
        p = p->next;    // Walk to end
    p->next = newNode;  // Link new node
}
```

Python — Insert at Tail

```
def insert_tail(head, val):
    new_node = Node(val)
    if head is None:
        return new_node
    p = head
    while p.next is not None:
        p = p.next      # Walk to end
    p.next = new_node   # Link new node
    return head
```

Complexity: O(N) because we must traverse to the end. A **tail pointer** can reduce this to O(1) — a common optimization for frequent tail insertions.

SLL Insertion in the Middle

📸 **Squeezing into a Photo:** Find the right spot between two friends, have the new person grab the right friend's hand first, then have the left friend let go of the right and grab the new person. Order is everything — do it wrong and someone is lost!

1

1. Find Position

Traverse with pointer `p` until reaching the node *before* the desired insertion point.

2

2. Link Right Side

Set `newNode->next = p->next`. New node now points to the node on the right.

3

3. Link Left Side

Set `p->next = newNode`. The left node now points to the new node. Done!

SLL Deletion

🚧 Bypassing a Closed Road

Deleting a node is like **re-routing traffic around a road closure**. You don't remove the road from the map entirely right away — you first find the detour: point the previous node directly to the node *after* the one being removed, then clean up.

Critical in C++: After re-routing, you must explicitly `delete` the removed node to free heap memory. Python handles this automatically via garbage collection.

Code: Delete by Value

```
// C++
void deleteNode(Node*& head, int val) {
 if (!head) return;
 if (head->data == val) {
 Node* tmp = head;
 head = head->next;
 delete tmp; return;
 }
 Node* p = head;
 while (p->next && p->next->data != val)
 p = p->next;
 if (p->next) {
 Node* tmp = p->next;
 p->next = tmp->next;
 delete tmp; // FREE MEMORY!
 }
}
```